

Healthcare Claims Backend: Complete Code Walkthrough

This document teaches the whole backend from zero. Every important line of code has a comment next to it explaining what it does, and every section explains the idea in plain words first. You do not need any backend experience to follow it.

1. Read this first: how to read the code

Before the real code, here are the few Python and FastAPI things that show up again and again. Once these click, the rest reads easily.

- **A function** is a named block of steps. `def submit_claim(...):` means "here is a step-by-step recipe called `submit_claim`".
- **A decorator** is the line starting with `@` written just above a function. It attaches extra behaviour. `@router.post("/submit")` means "run this function whenever someone sends a POST request to `/submit`".
- **A dictionary** is data written as `{"key": value}`. It is how we hold and send information, like `{"success": True, "code": "SUCCESS"}`.
- **return** ends the function and hands back a result to whoever called it.
- **if not something:** means "if this is empty, missing, or false". For example `if not member:` means "if we did not find a member".
- **None** means "nothing / not found". Our database functions return `None` when a record does not exist.
- **GET, POST, PUT, DELETE** are the four request types: GET reads, POST creates or sends, PUT updates, DELETE removes.

Keep these in mind and the comments in the code will make sense.

2. What this backend is (the big idea)

This backend is a small web service. An app, like the voice agent, sends it a request over the internet ("submit this claim", "is this member real"). The backend checks the data, reads or writes it to a database, and sends back a clean answer.

Three tools work together:

- **Python** is the language the code is written in.
- **FastAPI** is what turns our Python functions into web addresses (endpoints) like `/claims/submit`. Think of FastAPI as the receptionist who takes each request and hands it to the right person.

- **Firestore** is the database, where the data actually lives. It stores data as documents inside collections. A collection is like a folder or a table, a document is like one file or one row inside it.

The path is always the same: a request comes in, FastAPI passes it to the correct function, the function cleans the input, checks the rules, talks to Firestore, and returns an answer.

Every answer has the same four parts, which is what keeps the voice agent simple:

```
{
  "success": true,                // did it work: true or false
  "code": "CLAIM_SUBMITTED",      // a short label the agent reads
  "message": "Claim submitted successfully.", // a sentence for a human
  "data": { }                    // the actual content, if any
}
```

The agent only needs to look at `success` and `code` to know what happened next.

3. The path of one request (picture it once)

```
App / caller
|
| POST /claims/submit (data as JSON)
v
main.py           the receptionist, knows every endpoint
v
routers/claims.py the correct function runs here
|
|-- normalizers.py first, clean the messy input
|-- validators.py  then, check the rules
|-- firestore_service.py finally, read or write the database
v
Firestore         the data is saved or read
v
answer -> { success, code, message, data }
```

Every single feature in this backend is just this same path with different rules in the middle. Learn it once and everything else is a variation.

4. The files and what each one does

Everything lives under `healthcare-backend/app/`.

File	What it does
<code>main.py</code>	Starts the app and connects every router.
<code>database.py</code>	Opens the connection to Firestore.
<code>services/firestore_service.py</code>	The only file that reads and writes the database.
<code>routers/authentication.py</code>	Checks who a member is.
<code>routers/claims.py</code>	Submit, status, history, update, cancel a claim.
<code>routers/members.py</code>	Member profile lookup.
<code>routers/eligibility.py</code>	Is the coverage active.
<code>routers/benefits.py</code>	What the plan covers.
<code>routers/providers.py</code>	Find doctors and clinics.
<code>routers/policy.py</code>	Policy and plan details.
<code>routers/preauthorization.py</code>	Pre-approval status.
<code>models/request_models.py</code>	The shape of data coming in.
<code>models/response_models.py</code>	The shape of data going out.
<code>utils/validators.py</code>	The rules that check input.
<code>utils/normalizers.py</code>	The code that cleans input.
<code>seed/seed_firestore.py</code>	Fills the database with 10 sample members.
<code>Dockerfile</code>	Packages the app for the cloud.
<code>requirements.txt</code>	The list of libraries needed.

Why split it this way: database code sits in one place, rule code in another, cleaning code in another. Change one thing, touch one file. That is why the same small helpers (`normalize_member_id`, `validate_member_id`, `get_member`) appear again and again across the routers.

5. The app starter

File: `main.py` — the receptionist that starts everything.

This file creates the app and connects every router to it. After these lines, FastAPI knows that any address starting with `/claims` should go to the claims file, `/authentication` to the auth file, and so on.

```

# main.py
from fastapi import FastAPI

# Bring in each router (each is one group of endpoints, from its own file)
from app.routers.authentication import router as authentication_router
from app.routers.claims import router as claims_router
from app.routers.eligibility import router as eligibility_router
from app.routers.benefits import router as benefits_router
from app.routers.providers import router as providers_router
from app.routers.preauthorization import router as preauthorization_router
from app.routers.members import router as members_router
from app.routers.policy import router as policy_router

# Create the app itself. This "app" object IS the web service.
app = FastAPI(title="Healthcare Claims API", version="2.0.0")

# Attach each router. Now the app knows about all their endpoints.
app.include_router(authentication_router)
app.include_router(claims_router)
app.include_router(eligibility_router)
app.include_router(benefits_router)
app.include_router(providers_router)
app.include_router(preauthorization_router)
app.include_router(members_router)
app.include_router(policy_router)

# A tiny endpoint the cloud pings to check the app is alive.
@app.get("/health")
def health():
    return {"status": "Healthy", "service": "Healthcare Claims Backend"}

```

In short: `FastAPI(...)` builds the service, each `include_router` line plugs one group of features into it, and `/health` is just a heartbeat check. You only edit this file when you add a completely new router.

6. The database connection

File: `database.py` — opens the door to Firestore, once, for the whole app.

```

# database.py
from google.cloud import firestore

# Create ONE live connection to the database.
# Every other file imports this same "db" and reuses it.
db = firestore.Client()

```

That is the entire file. `firestore.Client()` connects to the database using the Google login of wherever the app runs (your laptop or the cloud), so no password is ever written in the code. Every file that needs the database just says `from app.database import db`.

7. The database layer (all reading and writing)

File: `services/firestore_service.py` — the only file allowed to touch the database.

Think of this file as the kitchen. The endpoints (the waiters) never cook themselves, they send orders here. If the database ever changes, you fix the kitchen once and every waiter keeps working. `@staticmethod` just means "a plain helper function grouped under this class, call it directly".

Reading members

```
@staticmethod
def get_member(member_id: str):
    # Go to the "members" folder, open the file named after this member ID.
    doc = db.collection("members").document(member_id).get()

    # If that file exists, hand back its contents as a dictionary.
    if doc.exists:
        return doc.to_dict()

    # Otherwise hand back None, meaning "no such member".
    return None
```

That `None` at the end is the whole trick of this system. The endpoint that called `get_member` checks "did I get None?" and, if so, decides which error to show. Read functions never decide the error, they just report what they found.

Reading claims

```
@staticmethod
def get_claim_by_member(member_id: str):
    docs = (db.collection("claims")
            .where("memberId", "==", member_id) # only this member's claims
            .order_by("submittedDate", direction=Query.DESCEENDING) # newest first
            .limit(1) # keep only the top one
            .stream())
    for doc in docs:
        return doc.to_dict() # return that single newest claim
    return None # member has no claims at all

@staticmethod
def get_claim_history(member_id: str):
    docs = (db.collection("claims")
            .where("memberId", "==", member_id) # same filter as above
            .order_by("submittedDate", direction=Query.DESCEENDING)
            .stream()) # but no limit this time
    # Turn every matching claim into a dictionary and return the whole list.
    return [doc.to_dict() for doc in docs]
```

Notice these two are almost the same query. The only real difference is `.limit(1)`. Status wants just the latest claim, history wants all of them. Same idea, one takes one row.

Making a new Claim ID

```
@staticmethod
def generate_claim_id():
    docs = db.collection("claims").stream()    # look at every existing claim
    highest = 1000                             # start counting from 1000

    for doc in docs:
        claim_id = doc.to_dict().get("claimId", "")    # read this claim's ID text
        if claim_id.startswith("CLM"):                # only the CLM... ones
            try:
                # Strip "CLM", turn the rest into a number, keep the biggest so far.
                highest = max(highest, int(claim_id.replace("CLM", "")))
            except ValueError:
                pass    # if it is not a clean number, skip it

    # The new ID is one more than the biggest existing one.
    return f"CLM{highest + 1}"
```

Plain version: it finds the largest claim number already in the database and adds one. Because the sample data goes up to CLM1010, the first brand new claim becomes **CLM1011**, then **CLM1012**, and so on.

Writing claims (submit, update, delete)

```
@staticmethod
def submit_claim(claim_data: dict):
    claim_id = FirestoreService.generate_claim_id()        # get the next ID
    current_time = datetime.now(timezone.utc).isoformat()  # timestamp for now

    # Stamp the extra fields onto the claim before saving.
    claim_data["claimId"] = claim_id
    claim_data["status"] = "Submitted"
    claim_data["submittedDate"] = datetime.now(timezone.utc).strftime("%Y-%m-%d")
    claim_data["createdAt"] = current_time
    claim_data["updatedAt"] = current_time
    claim_data["lastUpdated"] = current_time
    # Start a little history log so we can see the claim's journey later.
    claim_data["claimStatusHistory"] = [
        {"status": "Submitted", "timestamp": current_time}
    ]

    # .set(...) writes a NEW document into the claims folder.
    db.collection("claims").document(claim_id).set(claim_data)
    return claim_data


@staticmethod
def update_claim(claim_id: str, updated_data: dict):
    doc_ref = db.collection("claims").document(claim_id)
    doc = doc_ref.get()
    if not doc.exists:
        return None    # nothing to update, tell the caller "not found"

    current_time = datetime.now(timezone.utc).isoformat()
    existing_claim = doc.to_dict()

    # Figure out the status, and if it changed, add a new history entry.
    status = updated_data.get("status", existing_claim.get("status", "Processing"))
    history = existing_claim.get("claimStatusHistory", [])
    if not history or history[-1]["status"] != status:
        history.append({"status": status, "timestamp": current_time})

    updated_data["updatedAt"] = current_time
    updated_data["lastUpdated"] = current_time
    updated_data["claimStatusHistory"] = history

    # .update(...) changes ONLY the keys we pass, leaving the rest of the claim as is.
    doc_ref.update(updated_data)
    return doc_ref.get().to_dict()    # read it back and return the fresh version


@staticmethod
def delete_claim(claim_id: str):
    doc_ref = db.collection("claims").document(claim_id)
    doc = doc_ref.get()
    if not doc.exists:
        return False    # nothing there to delete

    doc_ref.delete()    # permanently remove the document
    return True
```

The one difference between `set` and `update` is worth remembering: `set` writes a whole new document, `update` changes only the fields you give it. And `delete` is final, once it runs, that claim is gone from the database for good.

Reading everything else

```
@staticmethod
def get_eligibility(member_id: str):
    # Open the eligibility file named after this member; return it or None.
    doc = db.collection("eligibility").document(member_id).get()
    return doc.to_dict() if doc.exists else None

@staticmethod
def get_benefits(member_id: str):
    doc = db.collection("benefits").document(member_id).get()
    return doc.to_dict() if doc.exists else None

@staticmethod
def get_provider(member_id: str):
    # Providers are filtered by a field, not opened by ID, so we query and take one.
    docs = (db.collection("providers")
            .where("memberId", "=", member_id).limit(1).stream())
    for doc in docs:
        return doc.to_dict()
    return None

@staticmethod
def search_provider(provider_name=None, city=None, state=None, zip_code=None):
    query = db.collection("providers") # start with all providers
    # Add a filter ONLY for each value the caller actually gave.
    if provider_name:
        query = query.where("providerNameLower", "=", provider_name.lower())
    if city:
        query = query.where("city", "=", city)
    if state:
        query = query.where("state", "=", state)
    if zip_code:
        query = query.where("zipCode", "=", zip_code)
    return [doc.to_dict() for doc in query.stream()] # return all matches

@staticmethod
def get_policy(member_id: str):
    doc = db.collection("policies").document(member_id).get()
    return doc.to_dict() if doc.exists else None

@staticmethod
def get_preauthorization(member_id: str):
    docs = (db.collection("preauthorizations")
            .where("memberId", "=", member_id).limit(1).stream())
    for doc in docs:
        return doc.to_dict()
    return None
```

They all follow one shape: look in a collection, hand back the document or `None`. The clever one is `search_provider`, it builds the query step by step, adding a filter only for the details the caller provided, so you can search by any mix of name, city, state, and zip.

8. How a member is validated (three layers)

This is the part people get confused about, because "validate a member" can mean three different things. The backend does all three, at different moments. Here they are from the weakest check to the strongest.

Step 0: Clean the input first

File: `utils/normalizers.py` — tidy up the messy input before any check.

```
def normalize_member_id(member_id: str) -> str:
    if not member_id:
        return "" # nothing came in, return empty

    member_id = member_id.strip().upper() # trim spaces, make it UPPERCASE
    member_id = (member_id
        .replace("MEMBER", "").replace("MEM", "") # drop any words
        .replace("-", "").replace("_", "").replace(" ", "")) # drop symbols

    digits = "".join(filter(str.isdigit, member_id)) # keep only the numbers
    if not digits:
        return "" # no numbers found, give up

    # Pad to 4 digits and stick "MEM" back on the front.
    return f"MEM{digits.zfill(4)}"
```

Why this exists: a voice agent might send "mem 1001", "MEMBER-1001", or just "1001". This throws away the words and symbols, keeps the digits, pads them to four, and adds "MEM". So every version becomes the same clean `MEM1001` before anything is checked. This is the golden rule of the whole backend: clean first, then check.

Layer 1: Is the ID even shaped right? (format check)

File: `utils/validators.py` — a quick pattern check, no database involved.

```
def validate_member_id(member_id: str) -> bool:
    # re.fullmatch checks the WHOLE text against a pattern.
    # r"MEM\d{4}" means: the letters MEM, then exactly 4 digits.
    return bool(re.fullmatch(r"MEM\d{4}", member_id))
```

This only asks "does it look like MEM plus four digits". It does not look anyone up. `MEM1001` passes, `MEM12` and `1001` fail. It is the cheapest check, so it runs first in almost every endpoint to reject obvious junk early.

Layer 2: Does this member actually exist? (existence check)

File: `routers/authentication.py` — the `verify_member` endpoint.

```
@router.get("/member/{member_id}", summary="Verify Member")
def verify_member(member_id: str):
    member_id = normalize_member_id(member_id)    # clean it first

    if not validate_member_id(member_id):         # then check the shape
        return {"success": False, "code": "INVALID_MEMBER_ID",
                "message": "Invalid Member ID."}

    member = FirestoreService.get_member(member_id) # now hit the database
    if not member:                                # got None back?
        return {"success": False, "code": "MEMBER_NOT_FOUND",
                "message": "Member not found."}

    # The member record exists, send back a few basic details.
    return {"success": True, "code": "SUCCESS",
            "message": "Member verified successfully.",
            "data": {"memberId": member.get("memberId"),
                     "memberName": member.get("memberName"),
                     "plan": member.get("plan"),
                     "policyNumber": member.get("policyNumber")}}
```

This is where we first touch the database. It confirms the account exists, but it does NOT prove the caller is that person. It is the "do we even have this member" step that the agent runs before asking for identity proof.

Layer 3: Is the caller truly this member? (real authentication)

File: `routers/authentication.py` — the `authenticate` endpoint. This is the strong one.

```
@router.post("/", summary="Authenticate Member")
def authenticate(request: AuthenticationRequest):
    # STEP 1: clean every field the caller sent.
    member_id = normalize_member_id(request.memberId)
    dob = normalize_date(request.dob)
    zip_code = normalize_zip(request.zipCode)
    ssn = normalize_ssn(request.last4SSN)

    # STEP 2: check each field is shaped correctly. First bad one wins.
    if not validate_member_id(member_id):
        return {"success": False, "code": "INVALID_MEMBER_ID",
              "message": "Invalid Member ID."}
    if not validate_date(dob):
        return {"success": False, "code": "INVALID_DOB",
              "message": "Invalid Date of Birth."}
    if not validate_past_date(dob): # DOB must be in the past
        return {"success": False, "code": "INVALID_DOB",
              "message": "Date of Birth cannot be today or a future date."}
    if not validate_zip(zip_code):
        return {"success": False, "code": "INVALID_ZIP",
              "message": "Invalid ZIP Code."}
    if not validate_ssn(ssn):
        return {"success": False, "code": "INVALID_SSN",
              "message": "Invalid Last 4 SSN."}

    # STEP 3: the member must exist in the database.
    member = FirestoreService.get_member(member_id)
    if not member:
        return {"success": False, "code": "MEMBER_NOT_FOUND",
              "message": "Member not found."}

    # STEP 4: THE ACTUAL IDENTITY CHECK.
    # All three details the caller gave must match what we have stored.
    if (member.get("dob") != dob or
        member.get("zipCode") != zip_code or
        member.get("last4SSN") != ssn):
        return {"success": False, "code": "AUTHENTICATION_FAILED",
              "message": "Authentication failed. Please verify your details."}

    # STEP 5: everything matched, they are who they say they are.
    return {"success": True, "code": "AUTHENTICATED",
          "message": "Authentication successful.",
          "data": {"memberId": member.get("memberId"),
                  "memberName": member.get("memberName"),
                  "plan": member.get("plan"),
                  "policyNumber": member.get("policyNumber"),
                  "email": member.get("email"),
                  "phone": member.get("phone")}}
```

Read step 4 closely, that is the heart of authentication. The caller had to supply date of birth, ZIP, and last 4 of SSN, and all three have to equal the stored values. If even one is wrong, it fails. This is what most people mean when they say "how do we validate a member".

So which one is the answer to "how do you validate a member"

What the question really means	The code that answers it
Is this a well-formed member ID?	<code>validate_member_id</code> (format only)
Do we have this member on file?	<code>verify_member</code> (calls <code>get_member</code>)
Is the caller truly this member?	<code>authenticate</code> (matches DOB + ZIP + SSN)

The honest one-line answer: we validate a member in three layers, format, existence, and identity. The identity layer is the real authentication, and it works by matching date of birth, ZIP code, and last 4 of SSN against the stored record.

9. Claims: submit a new claim

File: `routers/claims.py` — the `submit_claim` endpoint at `POST /claims/submit`.

The rules run in a fixed order and the first one that fails stops everything. Watch how each `if` guards one rule and returns its own clear error.

```

@router.post("/submit", summary="Submit New Claim")
def submit_claim(request: ClaimSubmissionRequest):
    # Clean the three text fields first.
    member_id = normalize_member_id(request.memberId)
    claim_type = normalize_claim_type(request.claimType)
    provider = normalize_provider(request.provider)

    # RULE 1: is the member ID shaped right?
    if not validate_member_id(member_id):
        return {"success": False, "code": "INVALID_MEMBER_ID",
                "message": "Invalid Member ID."}

    # RULE 2: does the member exist?
    member = FirestoreService.get_member(member_id)
    if not member:
        return {"success": False, "code": "MEMBER_NOT_FOUND",
                "message": "Member not found."}

    # RULE 3: is the claim type one we support?
    if not validate_claim_type(claim_type):
        return {"success": False, "code": "INVALID_CLAIM_TYPE",
                "message": "Unsupported claim type."}

    # RULE 4: is the amount a positive number?
    if request.amount <= 0:
        return {"success": False, "code": "INVALID_AMOUNT",
                "message": "Claim amount must be greater than zero."}

    # RULE 5: is the service date a real YYYY-MM-DD date?
    if not validate_date(request.serviceDate):
        return {"success": False, "code": "INVALID_SERVICE_DATE",
                "message": "Invalid service date format. Use YYYY-MM-DD."}

    # RULE 6: is that date today or in the past (never the future)?
    if not validate_not_future_date(request.serviceDate):
        return {"success": False, "code": "INVALID_SERVICE_DATE",
                "message": "Service date cannot be in the future."}

    # All rules passed. Build the claim record.
    claim = {
        "memberId": member_id,
        "claimType": claim_type,
        "provider": provider,
        "serviceDate": request.serviceDate,
        "diagnosisCode": request.diagnosisCode,
        "amount": request.amount,
        "status": "Submitted",
        "submittedDate": datetime.utcnow().strftime("%Y-%m-%d")
    }

    # Hand it to the kitchen, which assigns the new ID and saves it.
    result = FirestoreService.submit_claim(claim)

    return {"success": True, "code": "CLAIM_SUBMITTED",
            "message": "Claim submitted successfully.", "data": result}

```

The order matters: member ID, member exists, claim type, amount, date format, date not in the future. If a caller sends five things wrong, they only hear about the first one, fix it, and try

again. Only when all six pass does the claim actually get saved and a new Claim ID come back.

10. Claims: check status and history

File: `routers/claims.py` — `get_claim_status` and `claim_history` . Both only read.

```
@router.post("/status", summary="Get Claim Status")
def get_claim_status(request: ClaimStatusRequest):
    member_id = normalize_member_id(request.memberId)

    if not validate_member_id(member_id):          # shape check
        return {"success": False, "code": "INVALID_MEMBER_ID",
                "message": "Invalid Member ID."}

    member = FirestoreService.get_member(member_id) # member must exist
    if not member:
        return {"success": False, "code": "MEMBER_NOT_FOUND",
                "message": "Member not found."}

    claim = FirestoreService.get_claim_by_member(member_id) # latest claim
    if not claim:
        return {"success": False, "code": "CLAIM_NOT_FOUND",
                "message": "No claims found for this member."}

    return {"success": True, "code": "SUCCESS",
            "message": "Claim retrieved successfully.", "data": claim}

@router.get("/history/{member_id}", summary="Claim History")
def claim_history(member_id: str = Path(...)):
    member_id = normalize_member_id(member_id)
    claims = FirestoreService.get_claim_history(member_id) # ALL claims

    if not claims:
        return {"success": False, "code": "CLAIM_HISTORY_EMPTY",
                "message": "No claim history found."}

    return {"success": True, "code": "SUCCESS",
            "message": "Claim history retrieved successfully.",
            "totalClaims": len(claims), # how many claims there are
            "data": claims}
```

Status returns just the newest claim (or `CLAIM_NOT_FOUND` if there are none). History returns the full list plus a count. There is also a GET version of status at `/claims/status/{member_id}` that returns standard HTTP 400 or 404 errors instead of this friendly shape, handy for direct browser or code calls.

11. Claims: update a claim

File: routers/claims.py — `update_claim` at `PUT /claims/{claim_id}`.

The clever part here is that the caller only sends the fields they want to change. Each `if request.X is not None` block runs only for a field that was actually provided.

```
@router.put("/{claim_id}", summary="Update Claim")
def update_claim(claim_id: str, request: ClaimUpdateRequest):
    update_data = {}  # we will collect only the changed fields here

    # Only if a new claim type was sent: clean it, check it, keep it.
    if request.claimType is not None:
        claim_type = normalize_claim_type(request.claimType)
        if not validate_claim_type(claim_type):
            return {"success": False, "code": "INVALID_CLAIM_TYPE",
                    "message": "Unsupported claim type."}
        update_data["claimType"] = claim_type

    # Only if a new provider was sent.
    if request.provider is not None:
        update_data["provider"] = normalize_provider(request.provider)

    # Only if a new service date was sent: check format and not-future.
    if request.serviceDate is not None:
        if not validate_date(request.serviceDate):
            return {"success": False, "code": "INVALID_SERVICE_DATE",
                    "message": "Invalid service date format. Use YYYY-MM-DD."}
        if not validate_not_future_date(request.serviceDate):
            return {"success": False, "code": "INVALID_SERVICE_DATE",
                    "message": "Service date cannot be in the future."}
        update_data["serviceDate"] = request.serviceDate

    if request.diagnosisCode is not None:
        update_data["diagnosisCode"] = request.diagnosisCode

    if request.amount is not None:
        if request.amount <= 0:
            return {"success": False, "code": "INVALID_AMOUNT",
                    "message": "Claim amount must be greater than zero."}
        update_data["amount"] = request.amount

    # If the caller sent nothing to change, say so.
    if not update_data:
        return {"success": False, "code": "NO_UPDATE_FIELDS",
                "message": "No fields provided to update."}

    update_data["updatedAt"] = datetime.utcnow().strftime("%Y-%m-%d")
    updated = FirestoreService.update_claim(claim_id, update_data)

    # update_claim returns None when the claim ID does not exist.
    if not updated:
        return {"success": False, "code": "CLAIM_NOT_FOUND",
                "message": "Claim not found."}

    return {"success": True, "code": "CLAIM_UPDATED",
            "message": "Claim updated successfully.", "data": updated}
```

So the endpoint gathers only the provided fields into `update_data`, validates each as it goes, and passes them on. Whatever the caller did not send simply stays untouched on the claim.

12. Claims: cancel (delete) a claim

File: `routers/claims.py` — `delete_claim` at `DELETE /claims/{claim_id}`. Short but important.

```
@router.delete("/{claim_id}", summary="Cancel Claim")
def delete_claim(claim_id: str):
    # Ask the kitchen to delete it. Returns True if it removed something,
    # False if there was nothing with that ID.
    deleted = FirestoreService.delete_claim(claim_id)

    if not deleted:
        return {"success": False, "code": "CLAIM_NOT_FOUND",
                "message": "Claim not found."}

    return {"success": True, "code": "CLAIM_DELETED",
            "message": "Claim deleted successfully."}
```

Two things to keep in mind about cancel. First, it is a permanent delete, so the same claim cannot be cancelled twice, the second attempt gets `CLAIM_NOT_FOUND`. Second, it deletes purely by Claim ID and does not check that the claim belongs to the caller. Both points matter for testing and are called out in section 22.

13. Members router

File: `routers/members.py` — look up member profiles.

```
@router.get("/{member_id}", summary="Get Member Details")
def get_member_details(member_id: str = Path(...)):
    member_id = normalize_member_id(member_id)

    # This router raises HTTP errors instead of the friendly shape.
    if not validate_member_id(member_id):
        raise HTTPException(status_code=400, detail="Invalid Member ID.")

    member = FirestoreService.get_member(member_id)
    if not member:
        raise HTTPException(status_code=404, detail="Member not found.")

    return {"success": True, "code": "SUCCESS",
            "message": "Member details retrieved successfully.",
            "data": {"memberId": member.get("memberId"),
                     "memberName": member.get("memberName"),
                     "policyNumber": member.get("policyNumber"),
                     "plan": member.get("plan"),
                     "email": member.get("email"),
                     "phone": member.get("phone"),
                     "dob": member.get("dob"),
                     "zipCode": member.get("zipCode")}}

@router.get("/", summary="Get All Members")
def get_all_members():
    members = FirestoreService.get_all_members() # everyone
    return {"success": True, "code": "SUCCESS",
            "message": "Members retrieved successfully.",
            "count": len(members), "data": members}
```

First endpoint returns one member's full profile by ID, second returns everyone. Note the small difference from other routers: this one uses `raise HTTPException(...)`, so a bad ID gives a standard HTTP 400 and a missing member gives a 404, instead of the `success/code/message` shape.

14. The four lookup routers (eligibility, benefits, policy, pre-auth)

These four are the same recipe with different words swapped in. Learn the first one and you know all four. The recipe is: clean the ID, check its shape, confirm the member exists, read the record, return it or a not-found code.

File: `routers/eligibility.py` — is the coverage active. This is the template.

```
@router.post("/", summary="Check Eligibility")
def check_eligibility(request: EligibilityRequest):
    member_id = normalize_member_id(request.memberId)    # 1. clean

    if not validate_member_id(member_id):                # 2. shape check
        return {"success": False, "code": "INVALID_MEMBER_ID",
                "message": "Invalid Member ID."}

    member = FirestoreService.get_member(member_id)      # 3. member exists?
    if not member:
        return {"success": False, "code": "MEMBER_NOT_FOUND",
                "message": "Member not found."}

    eligibility = FirestoreService.get_eligibility(member_id) # 4. read record
    if not eligibility:
        return {"success": False, "code": "ELIGIBILITY_NOT_FOUND",
                "message": "Eligibility information not found."}

    return {"success": True, "code": "SUCCESS",          # 5. return it
            "message": "Eligibility retrieved successfully.", "data": eligibility}
```

File: `routers/benefits.py` — what the plan covers. Exact same recipe, it just calls `get_benefits` and uses the `BENEFITS_NOT_FOUND` code.

```
@router.post("/", summary="Get Member Benefits")
def get_benefits(request: BenefitsRequest):
    member_id = normalize_member_id(request.memberId)

    if not validate_member_id(member_id):
        return {"success": False, "code": "INVALID_MEMBER_ID",
                "message": "Invalid Member ID."}

    member = FirestoreService.get_member(member_id)
    if not member:
        return {"success": False, "code": "MEMBER_NOT_FOUND",
                "message": "Member not found."}

    benefits = FirestoreService.get_benefits(member_id)    # the only real difference
    if not benefits:
        return {"success": False, "code": "BENEFITS_NOT_FOUND",
                "message": "Benefits information not found."}

    return {"success": True, "code": "SUCCESS",
            "message": "Benefits retrieved successfully.", "data": benefits}
```

File: routers/policy.py — policy details, same recipe, plus a short summary endpoint that returns only the four most useful fields.

```
@router.post("/", summary="Get Policy Details")
def get_policy(request: PolicyRequest):
    member_id = normalize_member_id(request.memberId)
    if not validate_member_id(member_id):
        return {"success": False, "code": "INVALID_MEMBER_ID",
                "message": "Invalid Member ID."}
    member = FirestoreService.get_member(member_id)
    if not member:
        return {"success": False, "code": "MEMBER_NOT_FOUND",
                "message": "Member not found."}
    policy = FirestoreService.get_policy(member_id)
    if not policy:
        return {"success": False, "code": "POLICY_NOT_FOUND",
                "message": "Policy not found."}
    return {"success": True, "code": "SUCCESS",
            "message": "Policy retrieved successfully.", "data": policy}

@router.get("/summary/{member_id}", summary="Policy Summary")
def get_policy_summary(member_id: str = Path(...)):
    member_id = normalize_member_id(member_id)
    policy = FirestoreService.get_policy(member_id)
    if not policy:
        raise HTTPException(status_code=404, detail="Policy not found.")
    # Return just the highlights instead of the whole policy.
    return {"success": True, "code": "SUCCESS",
            "message": "Policy summary retrieved successfully.",
            "data": {"plan": policy.get("plan"),
                     "copay": policy.get("copay"),
                     "deductible": policy.get("deductible"),
                     "outOfPocket": policy.get("outOfPocket")}}
```

File: routers/preauthorization.py — pre-approval status. Same recipe once more, reading the **preauthorizations** records.

```
@router.post("/", summary="Get Pre-Authorization")
def get_preauthorization(request: PreAuthorizationRequest):
    member_id = normalize_member_id(request.memberId)
    if not validate_member_id(member_id):
        return {"success": False, "code": "INVALID_MEMBER_ID",
                "message": "Invalid Member ID."}
    member = FirestoreService.get_member(member_id)
    if not member:
        return {"success": False, "code": "MEMBER_NOT_FOUND",
                "message": "Member not found."}
    preauth = FirestoreService.get_preauthorization(member_id)
    if not preauth:
        return {"success": False, "code": "PREAUTHORIZATION_NOT_FOUND",
                "message": "Pre-authorization not found."}
    return {"success": True, "code": "SUCCESS",
            "message": "Pre-authorization retrieved successfully.", "data": preauth}
```

Once you truly get the eligibility one, the other three are just find-and-replace on the collection name and the not-found code. Nothing new is happening.

15. Providers router (the richest lookup)

File: `routers/providers.py` — find doctors and clinics several different ways.

This file can find providers by member, by search (name, city, state, zip), by NPI number, by state, by city, and by zip. One detail is critical and written right in the code as comments: the specific routes like `/search` and `/city` must be listed before the general `/member_id` route. If not, FastAPI reads the word "search" as if it were a member ID and the search endpoint breaks. This is a real bug that was fixed by ordering the routes correctly.

```

@router.post("/", summary="Provider Lookup")
def provider_lookup(request: ProviderLookupRequest):
    member_id = normalize_member_id(request.memberId)
    if not validate_member_id(member_id):
        return {"success": False, "code": "INVALID_MEMBER_ID",
                "message": "Invalid Member ID."}
    member = FirestoreService.get_member(member_id)
    if not member:
        return {"success": False, "code": "MEMBER_NOT_FOUND",
                "message": "Member not found."}
    provider = FirestoreService.get_provider(member_id)
    if not provider:
        return {"success": False, "code": "PROVIDER_NOT_FOUND",
                "message": "Provider not found."}
    return {"success": True, "code": "SUCCESS",
            "message": "Provider retrieved successfully.", "data": provider}

# SPECIFIC routes come FIRST, before /{member_id} below.
@router.get("/search", summary="Search Providers")
def search_provider(providerName: str | None = Query(None),
                   city: str | None = Query(None),
                   state: str | None = Query(None),
                   zipCode: str | None = Query(None)):
    # Clean each search value, then pass whichever were given to the query.
    providers = FirestoreService.search_provider(
        provider_name=normalize_provider(providerName) if providerName else None,
        city=city.title() if city else None,
        state=normalize_state(state) if state else None,
        zip_code=normalize_zip(zipCode) if zipCode else None)
    return {"success": True, "code": "SUCCESS",
            "count": len(providers), "data": providers}

@router.get("/npi/{npi}", summary="Search Provider by NPI")
def get_provider_by_npi(npi: str = Path(...)):
    npi = normalize_npi(npi) # keep only digits
    provider = FirestoreService.search_provider_by_npi(npi)
    if not provider:
        return {"success": False, "code": "PROVIDER_NOT_FOUND",
                "message": "Provider not found."}
    return {"success": True, "code": "SUCCESS",
            "message": "Provider retrieved successfully.", "data": provider}

@router.get("/network/{state}", summary="Providers by State")
def get_network_providers(state: str = Path(...)):
    providers = FirestoreService.search_provider(state=normalize_state(state))
    return {"success": True, "code": "SUCCESS",
            "count": len(providers), "data": providers}

@router.get("/city/{city}", summary="Providers by City")
def get_city_providers(city: str = Path(...)):
    providers = FirestoreService.search_provider(city=city.title())
    return {"success": True, "code": "SUCCESS",
            "count": len(providers), "data": providers}

@router.get("/zipcode/{zip_code}", summary="Providers by ZIP Code")

```

```
def get_zip_providers(zip_code: str = Path(...)):
    providers = FirestoreService.search_provider(zip_code=normalize_zip(zip_code))
    return {"success": True, "code": "SUCCESS",
            "count": len(providers), "data": providers}

# GENERAL route LAST, so it does not swallow /search, /city, and the rest.
@router.get("/{member_id}", summary="Get Provider")
def get_provider(member_id: str = Path(...)):
    member_id = normalize_member_id(member_id)
    if not validate_member_id(member_id):
        raise HTTPException(status_code=400, detail="Invalid Member ID.")
    provider = FirestoreService.get_provider(member_id)
    if not provider:
        raise HTTPException(status_code=404, detail="Provider not found.")
    return provider
```

Every search endpoint cleans its input its own way (city becomes Title Case, state becomes uppercase, zip becomes five digits) and then calls the same `search_provider` kitchen function with different filters. The route-order comments are not decoration, they are the actual fix that keeps `/search` working.

16. The input shapes

File: `models/request_models.py` — a contract for what each request must contain.

FastAPI reads these and automatically rejects a badly shaped request before your code even runs. `BaseModel` is the base every shape builds on.

```

from pydantic import BaseModel, Field

class AuthenticationRequest(BaseModel):
    memberId: str
    dob: str
    zipCode: str
    last4SSN: str

class ClaimStatusRequest(BaseModel):
    memberId: str

class ClaimSubmissionRequest(BaseModel):
    memberId: str
    claimType: str
    provider: str
    serviceDate: str
    diagnosisCode: str
    amount: float = Field(..., gt=0)  # gt=0 forces amount to be above zero here

class ClaimUpdateRequest(BaseModel):
    # Every field is optional (| None = None), which is what allows
    # the update endpoint to accept just the fields being changed.
    claimType: str | None = None
    provider: str | None = None
    serviceDate: str | None = None
    diagnosisCode: str | None = None
    amount: float | None = None

class EligibilityRequest(BaseModel):
    memberId: str

class BenefitsRequest(BaseModel):
    memberId: str

class ProviderLookupRequest(BaseModel):
    memberId: str

class PreAuthorizationRequest(BaseModel):
    memberId: str

class PolicyRequest(BaseModel):
    memberId: str

```

Two shapes are worth remembering. `ClaimSubmissionRequest` blocks a zero or negative amount right at the door with `Field(..., gt=0)`. `ClaimUpdateRequest` makes every field optional, which is exactly why the update endpoint can accept a request that changes only one field. The matching output shapes live in `models/response_models.py` and just describe the `success / code / message / data` structure.

17. All the rules

File: `utils/validators.py` — every yes-or-no check on the input.

These use regex, which are short text patterns. You do not need to read the regex, the comment next to each one tells you the rule.


```

import re
from datetime import datetime

# The only claim types the system accepts.
VALID_CLAIM_TYPES = {
    "Medical", "Dental", "Vision",
    "Pharmacy", "Mental Health", "Behavioral Health"
}

def validate_member_id(member_id: str) -> bool:
    return bool(re.fullmatch(r"MEM\d{4}", member_id))    # MEM + exactly 4 digits

def validate_date(date_string: str) -> bool:
    # Try to read it as YYYY-MM-DD. If that fails, it is not a valid date.
    try:
        datetime.strptime(date_string, "%Y-%m-%d")
        return True
    except ValueError:
        return False

def validate_zip(zip_code: str) -> bool:
    return bool(re.fullmatch(r"\d{5}", zip_code))        # exactly 5 digits

def validate_ssn(ssn: str) -> bool:
    return bool(re.fullmatch(r"\d{4}", ssn))             # exactly 4 digits

def validate_claim_type(claim_type: str) -> bool:
    return claim_type in VALID_CLAIM_TYPES              # must be in the allowed set

def validate_amount(amount: float) -> bool:
    try:
        return float(amount) > 0                       # must be above zero
    except Exception:
        return False

def validate_npi(npi: str) -> bool:
    return bool(re.fullmatch(r"\d{10}", npi))           # exactly 10 digits

def validate_policy_number(policy_number: str) -> bool:
    return bool(re.fullmatch(r"POL\d{6}", policy_number)) # POL + 6 digits

def validate_claim_id(claim_id: str) -> bool:
    return bool(re.fullmatch(r"CLM\d{4,}", claim_id))    # CLM + 4 or more digits

def validate_state(state: str) -> bool:
    return bool(re.fullmatch(r"[A-Z]{2}", state))        # 2 uppercase letters

def validate_email(email: str) -> bool:
    return bool(re.fullmatch(
        r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$", email))

def validate_phone(phone: str) -> bool:
    digits = re.sub(r"\D", "", phone)                   # strip non-digits
    return len(digits) >= 10                             # need at least 10

def validate_past_date(date_string: str) -> bool:
    """Date of birth: must be strictly BEFORE today."""
    try:
        d = datetime.strptime(date_string, "%Y-%m-%d").date()
        return d < datetime.utcnow().date()             # note the < (before)
    
```

```

except ValueError:
    return False

def validate_not_future_date(date_string: str) -> bool:
    """Service date: must be today OR earlier."""
    try:
        d = datetime.strptime(date_string, "%Y-%m-%d").date()
        return d <= datetime.utcnow().date() # note the <= (today ok)
    except ValueError:
        return False

```

The two date functions are the easy ones to mix up. `validate_past_date` is for date of birth and uses `<` (must be before today). `validate_not_future_date` is for service date and uses `<=` (today is allowed). One character of difference, two different rules.

Quick reference for the same rules:

Function	Rule	Passes	Fails
<code>validate_member_id</code>	MEM + 4 digits	MEM1001	MEM12
<code>validate_claim_id</code>	CLM + 4 or more digits	CLM1001	CLM12
<code>validate_date</code>	Real date, YYYY-MM-DD	2024-07-03	03-07-2024
<code>validate_past_date</code>	Real date before today	1985-04-15	today
<code>validate_not_future_date</code>	Real date today or earlier	today	tomorrow
<code>validate_zip</code>	Exactly 5 digits	10001	1000
<code>validate_ssn</code>	Exactly 4 digits	1234	12
<code>validate_claim_type</code>	One of the 6 allowed types	Medical	Neurology
<code>validate_amount</code>	Number above 0	7000	0
<code>validate_npi</code>	Exactly 10 digits	1234567893	12345
<code>validate_policy_number</code>	POL + 6 digits	POL100001	POL123
<code>validate_state</code>	2 uppercase letters	NY	ny
<code>validate_email</code>	Standard email	a@b.com	a@b
<code>validate_phone</code>	10+ digits	+1-212-555-1001	12345

18. All the cleaners

File: `utils/normalizers.py` — turn messy input into one clean, standard form.

```

import re
from dateutil import parser

def normalize_member_id(member_id: str) -> str:
    if not member_id:
        return ""
    member_id = member_id.strip().upper()           # trim + uppercase
    member_id = (member_id
        .replace("MEMBER", "").replace("MEM", "")   # drop words
        .replace("-", "").replace("_", "").replace(" ", "")) # drop symbols
    digits = "".join(filter(str.isdigit, member_id)) # keep digits only
    if not digits:
        return ""
    return f"MEM{digits.zfill(4)}"                  # pad to 4, add MEM

def normalize_date(date_value: str) -> str:
    if not date_value:
        return ""
    try:
        # dateutil understands almost any human date phrasing.
        parsed = parser.parse(date_value, dayfirst=True)
        return parsed.strftime("%Y-%m-%d")           # output strict format
    except Exception:
        return ""

def normalize_zip(zip_code: str) -> str:
    if not zip_code:
        return ""
    digits = re.sub(r"\D", "", zip_code)           # digits only
    return digits[:5] if len(digits) >= 5 else digits # first 5

def normalize_ssn(ssn: str) -> str:
    if not ssn:
        return ""
    digits = re.sub(r"\D", "", ssn)                 # digits only
    return digits[-4:] if len(digits) >= 4 else digits # last 4

def normalize_provider(provider: str) -> str:
    if not provider:
        return ""
    return " ".join(provider.strip().title().split()) # Title Case, tidy spaces

def normalize_claim_type(claim_type: str) -> str:
    if not claim_type:
        return ""
    # Map lowercase input to the exact accepted spelling.
    mapping = {
        "medical": "Medical", "dental": "Dental", "vision": "Vision",
        "pharmacy": "Pharmacy", "mental health": "Mental Health",
        "behavioral health": "Behavioral Health"
    }
    value = claim_type.strip().lower()
    return mapping.get(value, claim_type.strip().title())

def normalize_city(city: str) -> str:
    return city.strip().title() if city else ""

def normalize_state(state: str) -> str:
    return state.strip().upper() if state else ""

```

```
def normalize_npi(npi: str) -> str:
    return re.sub(r"\D", "", npi) if npi else "" # digits only

def normalize_policy_number(policy_number: str) -> str:
    if not policy_number:
        return ""
    return policy_number.strip().upper().replace("-", "").replace(" ", "")
```

The standout is `normalize_date`. It uses the `dateutil` library to understand almost any way a person might say a date ("3 july 2024", "07/03/2024") and rewrite it as the strict `2024-07-03` the rest of the system needs. The others mostly strip symbols and fix the casing.

Quick reference:

Function	Cleans up	Example
<code>normalize_member_id</code>	Adds MEM, pads to 4 digits	<code>mem 1001</code> -> <code>MEM1001</code>
<code>normalize_date</code>	Any date to YYYY-MM-DD	<code>3 july 2024</code> -> <code>2024-07-03</code>
<code>normalize_zip</code>	Digits only, first 5	<code>10001-1234</code> -> <code>10001</code>
<code>normalize_ssn</code>	Digits only, last 4	<code>xxx-xx-1234</code> -> <code>1234</code>
<code>normalize_provider</code>	Trim and Title Case	<code>city medical</code> -> <code>City Medical</code>
<code>normalize_claim_type</code>	Correct Title Case type	<code>medical</code> -> <code>Medical</code>
<code>normalize_city</code>	Title Case	<code>new york</code> -> <code>New York</code>
<code>normalize_state</code>	Uppercase	<code>ny</code> -> <code>NY</code>
<code>normalize_npi</code>	Digits only	<code>NPI 123</code> -> <code>123</code>
<code>normalize_policy_number</code>	Uppercase, remove symbols	<code>pol-100001</code> -> <code>POL100001</code>

The rule everywhere: clean first, then check, then use.

19. The sample data

File: `seed/seed_firestore.py` — fills an empty database so you have something to test.

Running this script creates data in seven collections:

Collection	Document ID is the
members	member ID (MEM1001 to MEM1010)
claims	claim ID (CLM1001 to CLM1010)
eligibility	member ID
benefits	member ID
providers	provider ID (PR1001 ...)
preauthorizations	authorization ID (PA10001 ...)
policies	member ID

Each member has exactly one claim: MEM1001 owns CLM1001 , and so on to MEM1010 / CLM1010 .

The main test member:

- Member ID: MEM1001 , Name: John Smith
- DOB: 1985-04-15 , ZIP: 10001 , Last 4 SSN: 1234
- Their one claim: CLM1001 , status "Under Review"

So to cancel or update a claim for John Smith, the only claim that exists out of the box is CLM1001 . Any other ID like CLM1021 will return CLAIM_NOT_FOUND unless a submit created it first.

20. Things to watch out for

Cancel is permanent, and CLM1021 is not in the sample data. Delete removes the document for good, so you cannot cancel the same claim twice. And the seed only creates CLM1001 to CLM1010 , new submitted claims start at CLM1011 . So testing cancel with CLM1021 fails unless it was created first. For repeatable testing, cancel a claim that exists (like CLM1001) or re-run the seed before each round.

A small typo in validators.py. The last line of validate_not_future_date currently reads return Falsealthcare instead of return False . If that branch runs (a bad service date reaches it), the app crashes. Change it to return False .

Cancel has no ownership check. It deletes by Claim ID without confirming the claim belongs to the caller. Worth tightening for a real system.

Provider route order. In providers.py , the specific routes (/search , /city , and so on) must come before /{member_id} , or FastAPI treats those words as member IDs and the endpoints break.

21. How to run it

1. Install Python 3.12.

2. In `healthcare-backend`, run `pip install -r requirements.txt`.
 3. Make sure Google credentials are available so Firestore can connect.
 4. Seed the database: `python seed/seed_firestore.py`.
 5. Start the server: `uvicorn app.main:app --reload --port 8080`.
 6. Open `http://localhost:8080/docs`. FastAPI builds an interactive page for every endpoint, the easiest way to try the API by hand.
-

22. The whole thing in one paragraph

A request hits `main.py`, which sends it to the right file in `routers/`. That function cleans the input with `normalizers.py`, checks it with `validators.py`, and reads or writes the database only through `firestore_service.py`. Every answer is `success / code / message / data`. A member is validated in three layers: format, existence, and identity, where identity is the real authentication that matches date of birth, ZIP, and last 4 of SSN. Claims can be submitted, read, updated, and cancelled. Test data comes from the seed: members `MEM1001` to `MEM1010`, each with one claim `CLM1001` to `CLM1010`. Cancel deletes for real, so re-seed between test rounds.